

PROYECTO MORTAL CODE

AYALA ALVAREZ NELSON ANDRES
OSORIO LOPEZ JUAN MANUEL
PALENCIA SANDOVAL DANIEL SANTIAGO
ROMERO SOTO MICHAELL STIVEN
SALAMANCA DIAZ JUAN CAMILO

TABLA DE CONTENIDO

INTRODUCCION AL PROBLEMA	4
Solución a la propuesta	4
¿Quién es el usuario?	5
¿Qué problema tiene?.....	5
¿En qué contexto usará la app?	5
Arquitectura de la solución planteada.....	5
Capas de la Arquitectura	5
Capa de Interfaz de Usuario (Frontend - Flutter)	5
Capa de Backend (Flask - Python).....	5
Capa de Servicios en la Nube (Firebase)	6
Capa de IA Generativa (Google Gemini).....	6
FLUJO GENERAL Y FUNCIONAMIENTO DEL SISTEMA.....	6
Tecnologías utilizadas	7
Users flows.....	8
FLUJO DE USUARIO	13
Historias de Usuario con Criterios de Aceptación:	13
WIREFRAMES.....	15
PROTOTIPO / MOCKUP	20
SCRUM.....	20
Formación de equipos	20
Sprints	21
Actas de Daily	21
Actas de Sprint Review	24
Actas de Sprint Retrospective.....	26
PRODUCT BACKLOG	28
Interfaz y Navegación del Juego	28
Sistema de Preguntas con IA	28
Sistema de Progreso	29
Desarrollo y Calidad	29
MODELO ENTIDAD-RELACION.....	30
MODELO RELACIONAL	30
DISEÑO DE ARQUITECTURA DE SOFTWARE	31
PRUEBAS UNITARIAS	32
Pruebas de Servicios HTTP con Mocktail.....	32
Pruebas de Widgets del Frontend.....	33
Pruebas Manuales con Thunder Client.....	34
EVIDENCIA.....	35

INTRODUCCION AL PROBLEMA

En la actualidad se evidencia que el aprendizaje de un lenguaje de programación como Python representa un desafío para muchos estudiantes principiantes debido a metodologías tradicionales basadas en teoría extensa, lo que puede llegar a ser tedioso, generar desmotivación y abandono temprano del aprendizaje. Muchas aplicaciones educativas existentes se enfocan únicamente en ejercicios técnicos sin ofrecer una experiencia atractiva o motivadora. Además, la falta de retroalimentación emocional o estímulos lúdicos limita la retención del conocimiento.

Solución a la propuesta

Se propone el desarrollo de un videojuego móvil educativo en el que el jugador avanza piso por piso dentro de un edificio abandonado resolviendo preguntas relacionadas con Python, generadas dinámicamente por Inteligencia Artificial (Google Gemini). Cada nivel aumenta progresivamente la dificultad, proporcionando retroalimentación inmediata sobre las respuestas correctas o incorrectas.

La combinación de gamificación, progresión narrativa, elementos de tensión e IA generativa busca mejorar la motivación, la concentración y la retención del aprendizaje, ofreciendo una experiencia educativa innovadora y entretenida. La aplicación estará estructurada en niveles que enseñan progresivamente:

La aplicación está estructurada en 8 niveles que enseñan progresivamente:

- Nivel 1: Variables en Python
- Nivel 2: Condicionales (if, elif, else)
- Nivel 3: Bucles (for, while, range)
- Nivel 4: Funciones (def, return, parámetros)
- Nivel 5: Listas (indexing, append, len, slicing)
- Nivel 6: Diccionarios (keys, values, get, items)
- Nivel 7: Clases (class, __init__, self, métodos)
- Nivel 8: Módulos (import, from, as, pip)

Cada nivel incluye:

- Identificar errores.
- Resolver pequeños retos lógicos.
- Preguntas con respuesta múltiple.

La aplicación incluirá:

- Pantallas interactivas por nivel
- Validación de las respuestas del usuario
- Retroalimentación inmediata
- Registro del progreso

¿Quién es el usuario?

Estudiantes de bachillerato, primeros semestres universitarios, personas autodidactas interesadas en aprender Python de forma interactiva o dinámica.

¿Qué problema tiene?

El usuario enfrenta principalmente los siguientes problemas:

- Dificultad para comprender conceptos abstractos como variables, condicionales, bucles y funciones.
- Falta de motivación al enfrentarse a métodos tradicionales basados en teoría y ejercicios repetitivos.
- Frustración al no entender errores de código.
- Sensación de que la programación es complicada o aburrida.

¿En qué contexto usará la app?

Utilizará la aplicación en dispositivos móviles durante tiempos libres, sesiones de estudio personal o como complemento de cursos académicos relacionados con programación.

Arquitectura de la solución planteada

La solución consiste en el desarrollo de un videojuego educativo para dispositivos móviles Android, con arquitectura de tres capas: frontend Flutter, backend Flask y servicios en la nube (Firebase + Google Gemini API).

Capas de la Arquitectura

Capa de Interfaz de Usuario (Frontend - Flutter)

Desarrollada con Flutter/Dart. Gestiona todas las pantallas del juego:

- Splash Screen con animación del logo
- Login y Registro con Firebase Authentication
- Home Screen con edificio animado (CustomPainter) y progreso visual
- Mapa de niveles tipo ruta con nodos desbloqueables
- Game Screen con vista en primera persona e imagen pixel art del computador IBM
- Pantalla de preguntas generadas por Gemini con sistema de 3 vidas
- Retroalimentación positiva y animación de muerte (Game Over)
- Configuración de audio y Créditos

Capa de Backend (Flask - Python)

API REST desarrollada con Flask que actúa como intermediario entre Flutter y Google Gemini:

- Endpoint POST /generar-pregunta: genera preguntas aleatorias según el nivel usando Gemini
- Endpoint POST /validar-respuesta: valida la respuesta del usuario y genera retroalimentación
- Endpoint GET /health: verifica el estado del servidor
- Flask-CORS habilitado para permitir peticiones desde el dispositivo móvil

- Sistema de fallback entre modelos Gemini para alta disponibilidad

Capa de Servicios en la Nube (Firebase)

- Firebase Authentication: registro e inicio de sesión con email y contraseña
- Cloud Firestore: almacenamiento del progreso del usuario por nivel en tiempo real

Capa de IA Generativa (Google Gemini)

La integración con Google Gemini es el núcleo educativo del sistema:

- Al tocar el computador IBM, Flutter envía una petición al backend Flask
- Flask construye un prompt específico según el nivel (ej: 'Genera una pregunta sobre variables en Python con 4 opciones...')
- Gemini genera una pregunta única, aleatoria y con explicación educativa en cada intento
- El sistema usa gemini-2.0-flash-lite como modelo principal (1000 req/día gratuitas) con fallback a gemini-2.0-flash y gemini-2.5-flash

FLUJO GENERAL Y FUNCIONAMIENTO DEL SISTEMA

El flujo principal de interacción dentro del videojuego se desarrolla de la siguiente manera:

1. El usuario inicia la app y ve la Splash Screen con el logo animado.
2. Se autentica con email y contraseña mediante Firebase Authentication.
3. Ve la Home Screen con el edificio abandonado; las ventanas iluminadas indican su progreso.
4. Accede al Mapa de Niveles y selecciona el nivel disponible.
5. Entra al nivel y ve la vista en primera persona con el computador IBM.
6. Al tocar el computador, Flutter envía petición al backend Flask que consulta Gemini.
7. Gemini genera una pregunta aleatoria de opción múltiple según el nivel.
8. El usuario responde; Flask valida y devuelve retroalimentación.
9. Al completar 5 preguntas correctas, el progreso se guarda en Firestore y se desbloquea el siguiente nivel.
10. Si el jugador agota sus 3 vidas, se muestra la animación de Game Over.

Este flujo permite integrar aprendizaje progresivo con mecánicas de juego que mantienen la motivación del usuario.

Tecnologías utilizadas

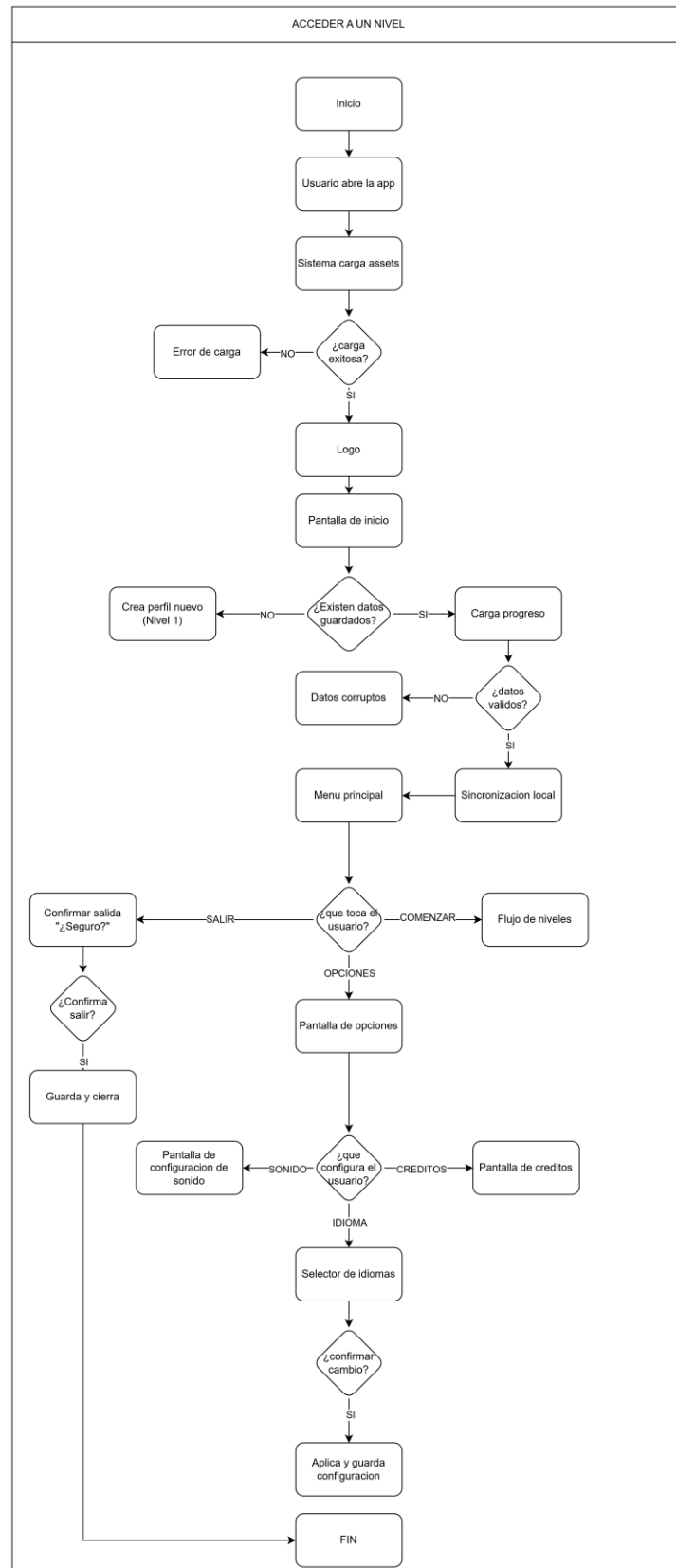
Para el desarrollo del videojuego se utilizarán las siguientes tecnologías:

Tecnología	Versión	Uso en el Proyecto
Flutter	3.41.9	Framework principal para el frontend móvil (Android)
Dart	3.11.5	Lenguaje de programación del frontend
Python	3.13	Lenguaje del backend Flask
Flask	3.1.3	Framework web para la API REST del backend
Flask-CORS	6.0.2	Habilita peticiones cross-origin desde el móvil
Google Gemini	2.0 / 2.5	IA generativa para crear preguntas educativas dinámicas
Firebase Auth	6.5.1	Autenticación de usuarios (registro e inicio de sesión)
Cloud Firestore	6.4.1	Base de datos en la nube para guardar progreso del usuario
firebase_core	4.9.0	Inicialización de Firebase en Flutter
http	1.6.0	Cliente HTTP para llamadas al backend Flask desde Flutter
flutter_animate	4.5.2	Animaciones declarativas en Flutter
google_fonts	8.1.0	Tipografías personalizadas (PressStart2P)
shared_preferences	2.5.5	Almacenamiento local de preferencias de configuración
mocktail	latest	Mocking para pruebas unitarias en Flutter
python-dotenv	1.2.2	Gestión de variables de entorno en Flask
Git / GitHub	2.54	Control de versiones y repositorio del proyecto
Android Studio	2025.3	SDK de Android para compilar y ejecutar en dispositivo real

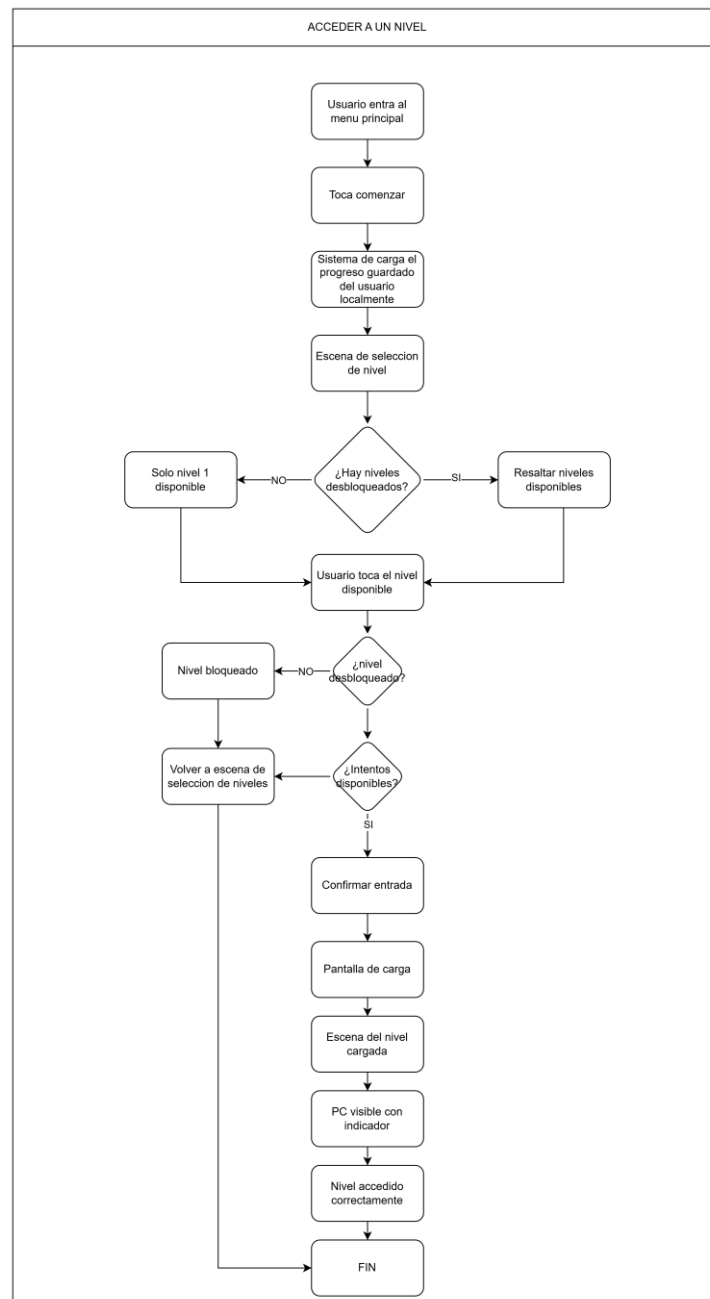
Users flows

A continuación, se presentan los users flows relacionados con el proyecto:

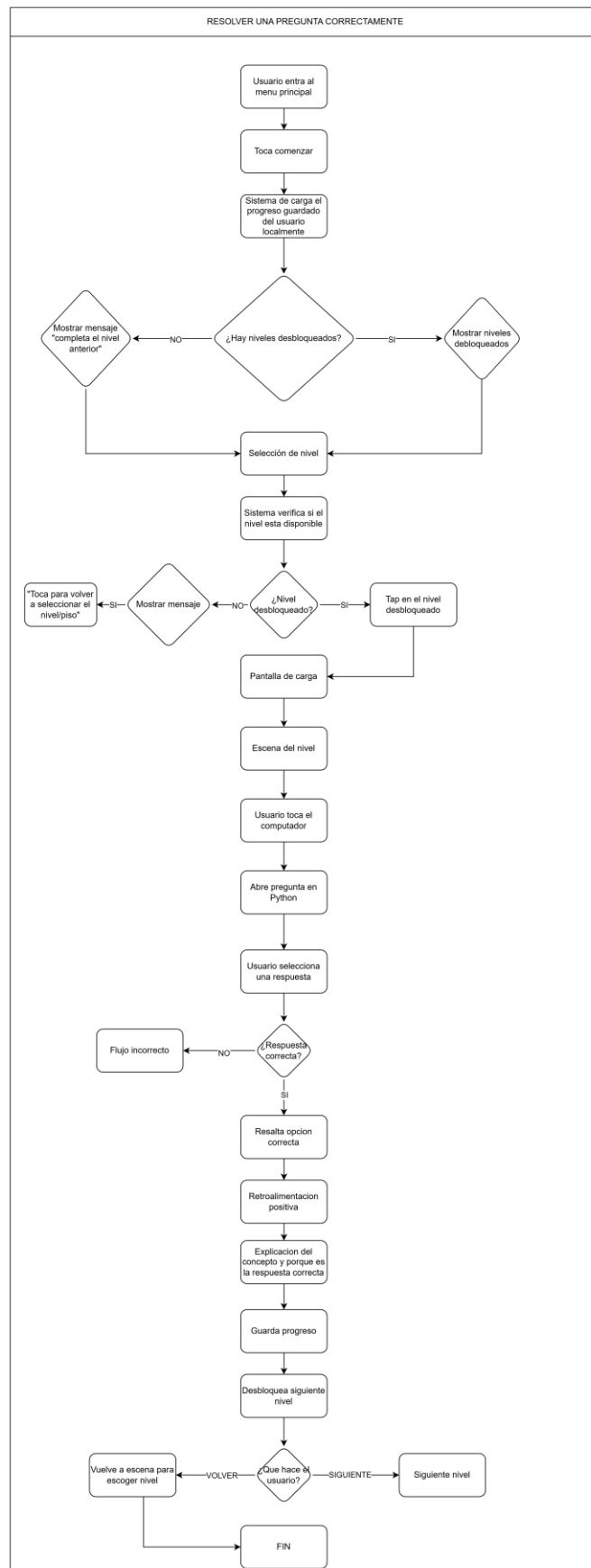
1. Iniciar el juego



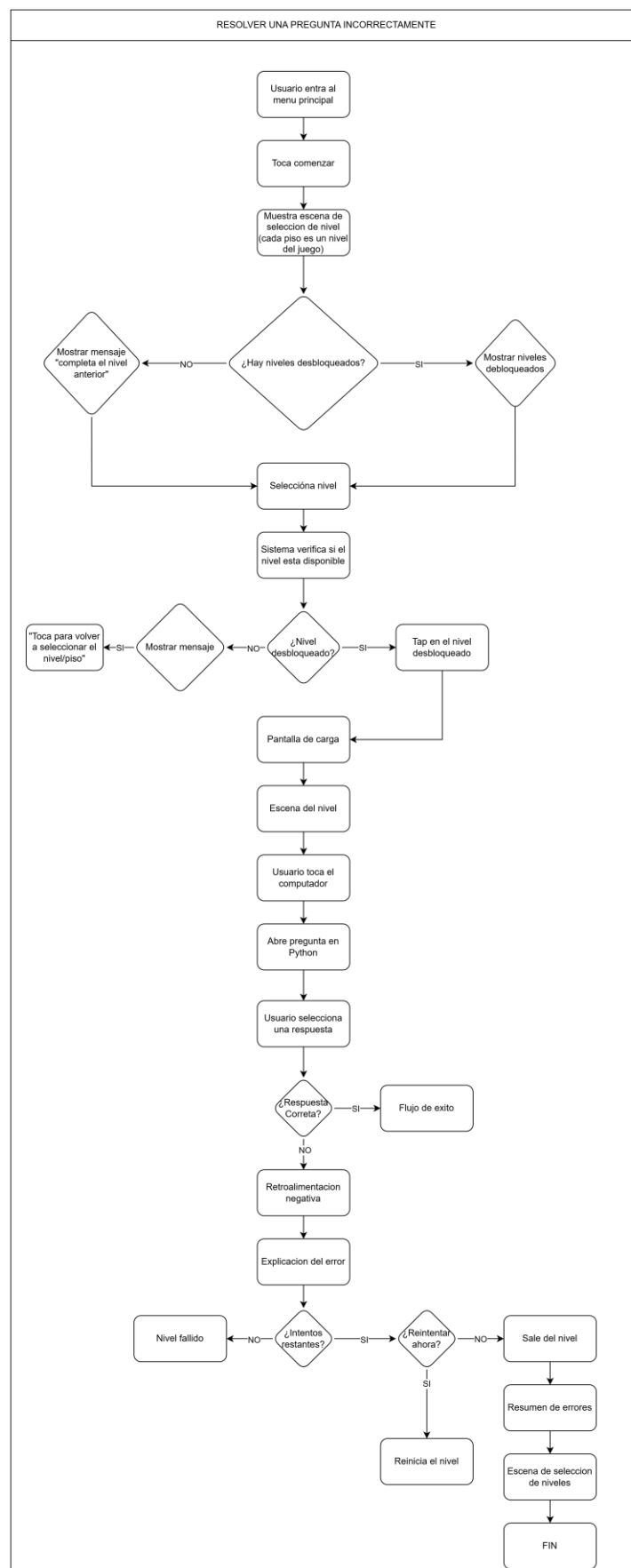
2. Acceder a un nivel



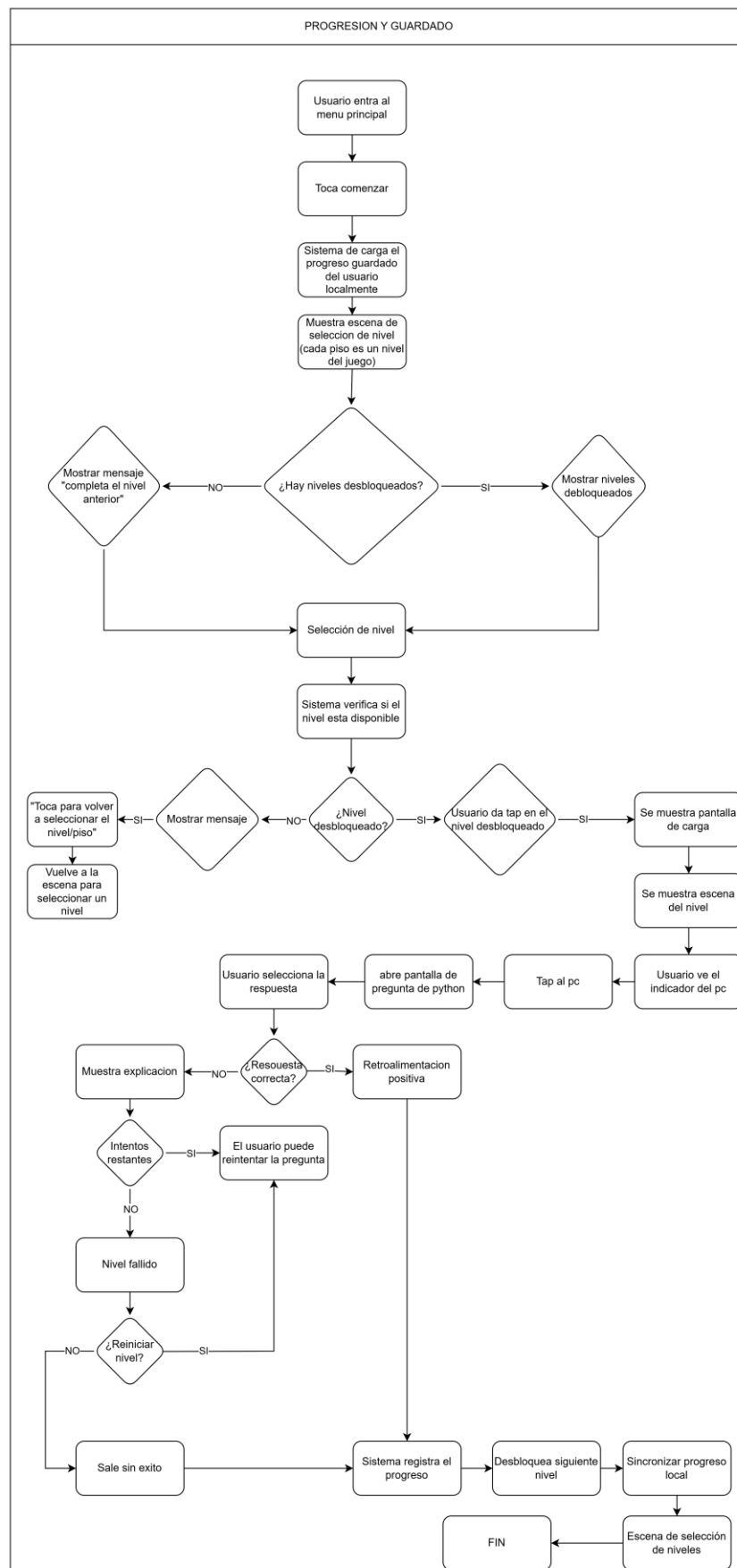
3. Resolver una pregunta correctamente



4. Responder incorrectamente



5. Progresión y guardado



FLUJO DE USUARIO

Historias de Usuario con Criterios de Aceptación:

HU-01: Iniciar o continuar partida

ID	HU-01
Rol	Como estudiante que desea aprender Python, quiero iniciar o continuar una partida desde el menú principal para acceder rápidamente a mi progreso en el juego.
Estimación	3 Story Points
Prioridad	Alta
Sprint	Sprint 1
Criterio de aceptación	El sistema carga el progreso guardado desde Firestore y muestra el edificio con el piso actual iluminado. Si no hay datos guardados, inicia desde el nivel 1.

HU-02: Autenticación con Firebase

ID	HU-02
Rol	Como usuario nuevo, quiero registrarme con mi correo y contraseña para tener una cuenta personal que guarde mi progreso en la nube.
Estimación	3 Story Points
Prioridad	Alta
Sprint	Sprint 1
Criterio de aceptación	El usuario puede registrarse e iniciar sesión con email/contraseña mediante Firebase Authentication. Los errores (correo ya registrado, contraseña débil) se muestran en pantalla.

HU-03: Seleccionar nivel en el mapa

ID	HU-03
Rol	Como jugador, quiero ver el mapa de niveles para seleccionar el nivel disponible y acceder al juego.
Estimación	5 Story Points
Prioridad	Alta
Sprint	Sprint 2
Criterio de aceptación	El mapa muestra 8 niveles en forma de ruta. Los niveles bloqueados muestran un candado. Solo el nivel actual o completados son seleccionables.

HU-04: Generar pregunta con IA Gemini

ID	HU-04
Rol	Como jugador, quiero resolver una pregunta generada por IA al tocar el computador IBM dentro del nivel para avanzar en el juego.
Estimación	8 Story Points
Prioridad	Alta
Sprint	Sprint 3
Criterio de aceptación	Al tocar el computador, Flutter envía el nivel al backend Flask que consulta Gemini. Se muestra una pregunta de opción múltiple única y aleatoria en español sobre el tema del nivel.

HU-05: Sistema de vidas y Game Over

ID	HU-05
Rol	Como jugador, quiero recibir retroalimentación visual cuando respondo incorrectamente y ver la animación de Game Over si agoto mis 3 vidas.
Estimación	5 Story Points
Prioridad	Alta
Sprint	Sprint 4
Criterio de aceptación	El sistema muestra 3 corazones. Al responder mal se pierde uno. Al perder todos se muestra la pantalla de Game Over con la explicación de la respuesta correcta y opción de reintentar.

HU-06: Guardar progreso en Firestore

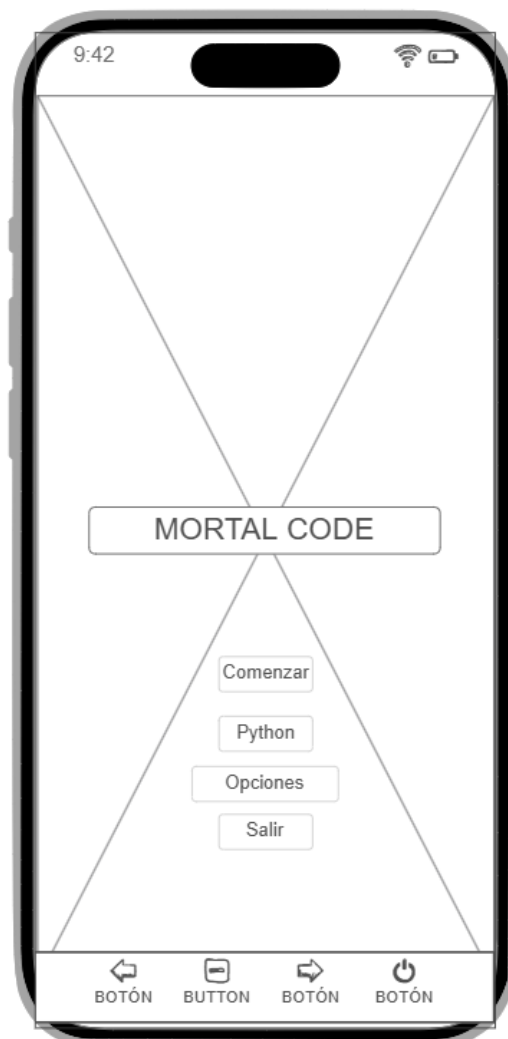
ID	HU-06
Rol	Como jugador, quiero que mi progreso se guarde automáticamente en la nube al completar un nivel para continuar desde donde lo dejé.
Estimación	5 Story Points
Prioridad	Alta
Sprint	Sprint 4
Criterio de aceptación	Al completar 5 preguntas correctas en un nivel, el progreso se guarda en Cloud Firestore. El edificio actualiza las ventanas iluminadas. Se puede reiniciar el progreso desde la Home Screen.

HU-07: Configuración de audio y créditos

ID	HU-07
Rol	Como jugador, quiero poder activar o desactivar el sonido y ver los créditos del juego desde la pantalla de configuración.
Estimación	3 Story Points
Prioridad	Media
Sprint	Sprint 5
Criterio de aceptación	La pantalla de configuración tiene switches para efectos de sonido y música. Las preferencias se guardan con SharedPreferences. El botón de créditos lleva a la pantalla de créditos con el equipo y universidad.

WIREFRAMES

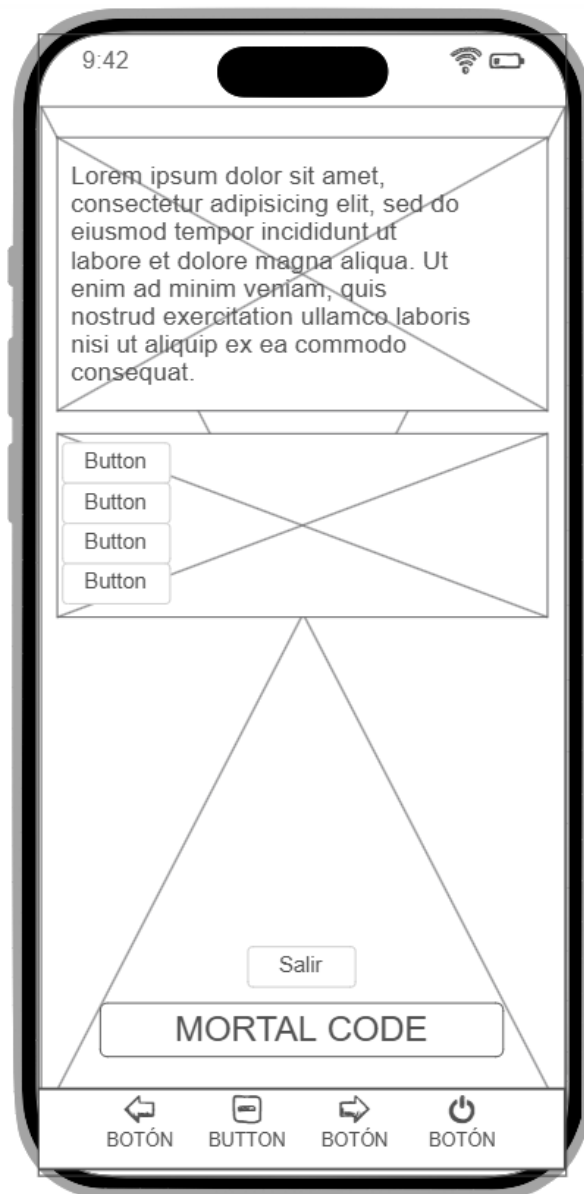
Pantalla de inicio:



Niveles:



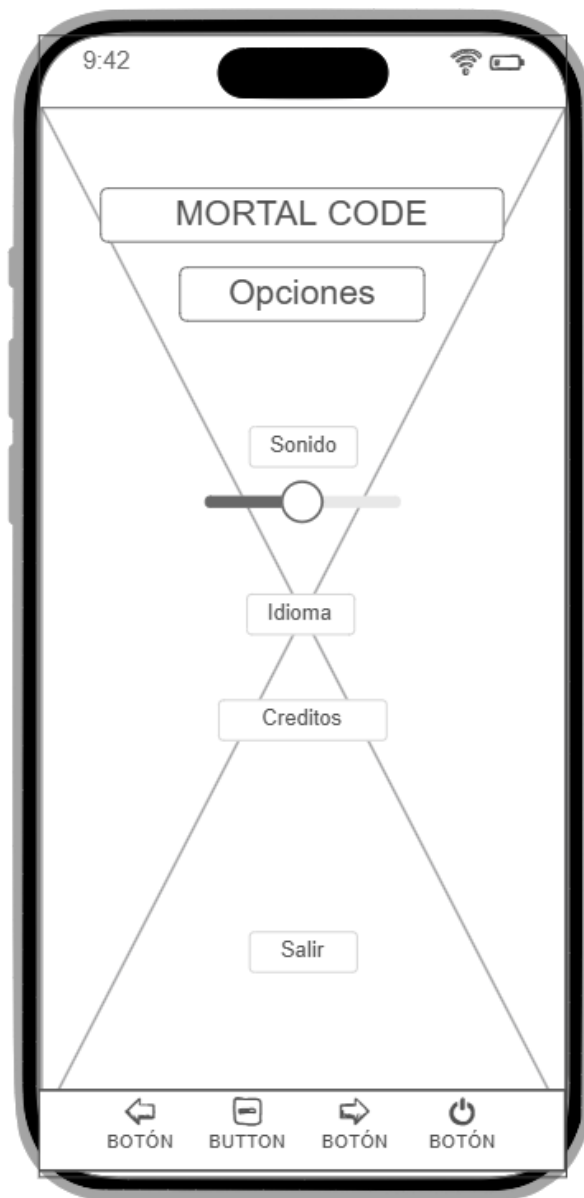
Juego Nivel 1:



Error:



Opciones:



PROTOTIPO / MOCKUP

Con el objetivo de visualizar la estructura y funcionamiento general del videojuego propuesto, se desarrolló un prototipo interactivo utilizando la herramienta NinjaMock. Este prototipo permite representar de manera conceptual la interfaz de usuario y el flujo de navegación dentro de la aplicación móvil antes de iniciar el desarrollo completo del sistema.

El mockup presenta una simulación de las principales pantallas que conformarán el videojuego, permitiendo comprender cómo el usuario interactuará con la aplicación y cómo se organizarán los elementos visuales dentro de la experiencia de juego.

Dentro del prototipo se incluyen las siguientes pantallas principales:

- Pantalla de inicio del juego con la visualización del edificio.
- Pantalla de selección de niveles representados por los pisos del edificio.
- Pantalla de un nivel básico con una pregunta introductoria de Python.
- Pantalla de carga del juego.
- Pantalla de error cuando el usuario responde incorrectamente.
- Pantalla de respuesta correcta con su respectiva retroalimentación educativa.
- Pantalla de opciones del juego.
- Pantalla de créditos.

Este prototipo permite validar la experiencia de usuario (UX), la navegación entre pantallas y la organización general de la interfaz, facilitando la identificación de posibles mejoras antes de la etapa de desarrollo del videojuego.

A continuación, se presenta el enlace al prototipo interactivo desarrollado en NinjaMock:

<https://ninjamock.com/s/GVNQWZx>

SCRUM

Formación de equipos

Rol	Nombre
Scrum Master	Nelson Andrés Ayala Álvarez
Product Owner	Nelson Andrés Ayala Álvarez
Programador Principal	Nelson Ayala, Daniel Palencia
Diseño y Documentación	Nelson Ayala, Juan Osorio, Daniel Palencia, Michael Romero, Camilo Salamanca
Testing y QA	Nelson Ayala, Juan Osorio, Michael Romero, Camilo Salamanca

Sprints

Sprint	Objetivo	Entregables principales
Sprint 1	Base del proyecto y autenticación	Proyecto Flutter creado, Firebase configurado, Login/Registro funcional, Splash Screen con logo
Sprint 2	Navegación y pantallas principales	Home Screen con edificio CustomPainter, Mapa de niveles, Navegación entre pantallas
Sprint 3	Backend Flask e integración Gemini	API REST con Flask, endpoints /generar-pregunta y /validar-respuesta, pruebas con Thunder Client
Sprint 4	Sistema de juego completo	Game Screen con imagen IBM, sistema de 3 vidas, botón continuar, retroalimentación, Game Over
Sprint 5	Progreso y persistencia	Guardado en Firestore, progreso visual en edificio, reinicio de progreso, configuración de audio
Sprint 6	Assets, pruebas y entrega	Logo y imagen IBM integrados, 19 pruebas unitarias pasando, GitHub, documentación final

Actas de Daily

Acta Daily Scrum — Sprint 1 - Día 1

Fecha: Semana 1 – Martes 12 de mayo 2026

Participantes: Nelson Ayala, Daniel Palencia, Michael Romero, Juan Salamanca, Juan Osorio

Temas tratados:

- ¿Qué hice ayer? Reunión inicial de planificación del proyecto. Definición del stack tecnológico.
- ¿Qué haré hoy? Configurar el entorno de desarrollo: Flutter, Python, Android Studio.
- ¿Hay impedimentos? Algunos miembros no tienen Flutter en el PATH — se resuelve agregando la ruta manualmente.

Acuerdos y compromisos:

- Nelson lidera la configuración del entorno en Windows.
- Se confirma el uso de Flutter + Flask + Firebase + Gemini como stack definitivo.
- Se descarta Godot Engine y GDScript en favor de Flutter/Dart por mayor control y mejor integración con Firebase.

Acta Daily Scrum — Sprint 1 - Día 3

Fecha: Semana 1 – Jueves 14 de mayo 2026

Participantes: Nelson Ayala, Daniel Palencia, Michael Romero, Juan Salamanca, Juan Osorio

Temas tratados:

- ¿Qué hice ayer? Configuración de Firebase: Authentication y Firestore habilitados.
- ¿Qué haré hoy? Crear el proyecto Flutter con flutter create y agregar dependencias.
- ¿Hay impedimentos? Error de symlinks en Windows — se resuelve activando modo desarrollador.

Acuerdos y compromisos:

- Se ejecuta flutter pub add con todas las dependencias del proyecto.
- Se configura FlutterFire CLI y se genera firebase_options.dart.
- La estructura de carpetas queda definida: screens/, services/, models/, assets/.

Acta Daily Scrum — Sprint 2 - Día 7

Fecha: Semana 2 – Lunes 18 de mayo 2026

Participantes: Nelson Ayala, Daniel Palencia, Michael Romero, Juan Salamanca, Juan Osorio

Temas tratados:

- ¿Qué hice ayer? Desarrollo de la Home Screen con CustomPainter para el edificio.
- ¿Qué haré hoy? Mejorar el CustomPainter: agregar ladrillos, grietas, luna, estrellas y edificios de fondo.
- ¿Hay impedimentos? El suelo del edificio tapa la primera ventana — se ajustan los valores de altura del edificio.

Acuerdos y compromisos:

- Se aumenta edificioAlto a 0.88 y se agrega alturaPiso adicional para que todas las ventanas queden dentro.
- Se implementa el efecto de parpadeo (flicker) para la ventana del nivel actual.
- Se agrega el mapa de niveles con la lógica de desbloqueo.

Acta Daily Scrum — Sprint 3 - Día 8

Fecha: Semana 2 – Martes 19 de mayo de 2026

Participantes: Nelson Ayala, Daniel Palencia, Michael Romero, Juan Salamanca, Juan Osorio

Temas tratados:

- ¿Qué hice ayer? Creación del backend Flask con entorno virtual Python.
- ¿Qué haré hoy? Implementar el endpoint /generar-pregunta conectado a Gemini.
- ¿Hay impedimentos? La librería google-generativeai está deprecada — se migra a google-genai.

Acuerdos y compromisos:

- Se actualiza el backend para usar from google import genai con el cliente nuevo.
- Se configura el modelo gemini-2.5-flash y se prueba con Thunder Client.
- Los endpoints /generar-pregunta y /validar-respuesta responden correctamente (200 OK).

Acta Daily Scrum — Sprint 4 - Día 10

Fecha: Semana 2 – Jueves 21 de mayo 2026

Participantes: Nelson Ayala, Daniel Palencia, Michael Romero, Juan Salamanca, Juan Osorio

Temas tratados:

- ¿Qué hice ayer? Integración de la imagen del computador IBM como asset de Flutter.
- ¿Qué haré hoy? Implementar la Game Screen con la imagen como fondo y el efecto de zoom al tocar.
- ¿Hay impedimentos? El GestureDetector queda detrás del overlay — se mueve al wrapper del Stack.

Acuerdos y compromisos:

- Se envuelve el Stack con GestureDetector para capturar correctamente los taps.
- Se implementa AnimatedScale para el efecto de acercamiento al computador.
- El sistema de 3 vidas y 5 preguntas por nivel queda funcional.

Acta Daily Scrum — Sprint 5 - Día 15

Fecha: Semana 3 – Martes 26 de Mayo 2026

Participantes: Nelson Ayala, Daniel Palencia, Michael Romero, Juan Salamanca, Juan Osorio

Temas tratados:

- ¿Qué hice ayer? Implementación del guardado de progreso en Firestore.
- ¿Qué haré hoy? Verificar que el edificio actualiza las ventanas al regresar del mapa.
- ¿Hay impedimentos? El progreso no se actualiza automáticamente en la Home Screen al volver del mapa.

Acuerdos y compromisos:

- Se agrega .then((_) => _cargarProgreso()) al botón COMENZAR para recargar el progreso al volver.
- Se verifica en Firebase Console que el documento del usuario se crea correctamente.
- Se implementa el botón de reinicio de progreso con confirmación.

Acta Daily Scrum — Sprint 6 - Día 18

Fecha: Semana 3 – Viernes 28 de mayo de 2026

Participantes: Nelson Ayala, Daniel Palencia, Michael Romero, Juan Salamanca, Juan Osorio

Temas tratados:

- ¿Qué hice ayer? Implementación de pruebas unitarias con Mocktail para GameService.
- ¿Qué haré hoy? Implementar pruebas de widgets para ConfigScreen y CreditsScreen.
- ¿Hay impedimentos? flutter_animate deja timers pendientes en los tests — se resuelve con pump() explícito.

Acuerdos y compromisos:

- Se agregan await tester.pump(Duration(seconds: 1)) dos veces para esperar las animaciones.
- 19 pruebas en total pasan: 6 de servicios HTTP + 13 de widgets.
- Se sube el proyecto a GitHub con README, requirements.txt y .gitignore correctamente configurado.

Actas de Sprint Review

Acta Sprint Review — Sprint 1

Fecha: Fin de Semana 1

Participantes: Nelson Ayala, Daniel Palencia, Michael Romero, Juan Salamanca, Juan Osorio

Temas tratados:

- Demostración de la Splash Screen con animación del logo Mortal Code.
- Demostración del Login y Registro funcional con Firebase Authentication.
- Revisión de la estructura de carpetas del proyecto Flutter.

Acuerdos y compromisos:

- El Login funciona correctamente — usuario aparece en Firebase Console.
- La Splash Screen navega automáticamente al Login después de 4 segundos.
- Para el Sprint 2 se priorizará el edificio y el mapa de niveles.

Velocidad del Sprint: 15 Story Points completados de 15 planificados.

Acta Sprint Review — Sprint 2

Fecha: Fin de Semana 2

Participantes: Nelson Ayala, Daniel Palencia, Michael Romero, Juan Salamanca, Juan Osorio

Temas tratados:

- Demostración del edificio con CustomPainter: ladrillos, ventanas, luna y estrellas.
- Demostración del mapa de niveles con los 8 nodos y lógica de desbloqueo.
- Revisión de la navegación entre todas las pantallas.

Acuerdos y compromisos:

- El edificio luce visualmente atractivo con la temática de terror.
- La navegación entre pantallas funciona correctamente.
- Para el Sprint 3 se implementará el backend Flask y la integración con Gemini.

Velocidad del Sprint: 18 Story Points completados de 18 planificados.

Acta Sprint Review — Sprint 3**Fecha: Fin de Semana 2**

Participantes: Nelson Ayala, Daniel Palencia, Michael Romero, Juan Salamanca, Juan Osorio

Temas tratados:

- Demostración de los 3 endpoints de Flask probados con Thunder Client.
- Demostración de generación de pregunta de nivel 1 con Gemini en español.
- Revisión del sistema de fallback entre modelos Gemini.

Acuerdos y compromisos:

- El backend genera preguntas correctamente y valida respuestas con retroalimentación.
- Se detecta error 429 (cuota agotada) — se implementa fallback a modelos con mayor cuota.
- Para el Sprint 4 se conectará Flutter con el backend y se construirá la Game Screen completa.

Velocidad del Sprint: 21 Story Points completados de 21 planificados.

Acta Sprint Review — Sprint 4**Fecha: Fin de Semana 2**

Participantes: Nelson Ayala, Daniel Palencia, Michael Romero, Juan Salamanca, Juan Osorio

Temas tratados:

- Demostración del juego completo corriendo en el Realme 6 Pro vía ADB.
- Demostración del flujo: tocar computador → cargar pregunta → responder → retroalimentación → continuar.
- Demostración de Game Over con animación de calavera.

Acuerdos y compromisos:

- El juego corre en dispositivo real Android 11 satisfactoriamente.
- El sistema de vidas y el botón CONTINUAR funcionan correctamente.
- Para el Sprint 5 se implementará el guardado en Firestore y la configuración de audio.

Velocidad del Sprint: 18 Story Points completados de 18 planificados.

Acta Sprint Review — Sprint 5

Fecha: Fin de Semana 3

Participantes: Nelson Ayala, Daniel Palencia, Michael Romero, Juan Salamanca, Juan Osorio

Temas tratados:

- Demostración del progreso guardado en Firestore visible en Firebase Console.
- Demostración del edificio actualizando las ventanas al completar un nivel.
- Demostración de la pantalla de configuración con switches de audio.

Acuerdos y compromisos:

- El progreso se guarda y carga correctamente desde la nube.
- El reinicio de progreso funciona con mensaje de confirmación.
- Para el Sprint 6 se implementarán las pruebas unitarias y se subirá a GitHub.

Velocidad del Sprint: 18 Story Points completados de 18 planificados.

Acta Sprint Review — Sprint 6

Fecha: Fin de Semana 3

Participantes: Nelson Ayala, Daniel Palencia, Michael Romero, Juan Salamanca, Juan Osorio

Temas tratados:

- Demostración de 19 pruebas unitarias pasando (6 Mocktail + 13 widgets).
- Demostración del repositorio GitHub con README y requirements.txt.
- Revisión final del proyecto completo corriendo en dispositivo real.

Acuerdos y compromisos:

- El proyecto está completo y funcional.
- GitHub está configurado correctamente como repositorio privado.
- La documentación final se completa para entrega.

Velocidad del Sprint: 18 Story Points completados de 18 planificados.

Actas de Sprint Retrospective

Acta Sprint Retrospective — Sprint 1 y 2

Fecha: Fin de Semana 2

Participantes: Nelson Ayala, Daniel Palencia, Michael Romero, Juan Salamanca, Juan Osorio

Temas tratados:

- ¿Qué salió bien? La configuración del entorno fue más rápida de lo esperado.
- ¿Qué salió mal? El PATH de Flutter y Dart no se configuró automáticamente en Windows.
- ¿Qué mejorar? Documentar los comandos de configuración del entorno desde el inicio.

Acuerdos y compromisos:

- Se documenta el proceso de configuración del PATH para facilitar la presentación en la universidad.
- Se establece convención de nombres para archivos y carpetas del proyecto.
- Se acuerda hacer commits frecuentes a GitHub para no perder trabajo.

Acta Sprint Retrospective — Sprint 3 y 4

Fecha: Fin de Semana 3

Participantes: Nelson Ayala, Daniel Palencia, Michael Romero, Juan Salamanca, Juan Osorio

Temas tratados:

- ¿Qué salió bien? La integración con Gemini fue más sencilla de lo esperado con la nueva librería google-gemini.
- ¿Qué salió mal? El límite de cuota gratuita de Gemini (20 req/día) se agotó durante las pruebas.
- ¿Qué mejorar? Usar gemini-2.0-flash-lite (1000 req/día) como modelo principal para no agotar la cuota.

Acuerdos y compromisos:

- Se implementa sistema de fallback entre modelos Gemini para garantizar disponibilidad.
- Se acuerda no hacer pruebas excesivas antes de la presentación para conservar la cuota.
- Se implementa el botón CONTINUAR para dar tiempo al usuario de leer la retroalimentación.

Acta Sprint Retrospective — Sprint 5 y 6

Fecha: Fin de Semana 3

Participantes: Nelson Ayala, Daniel Palencia, Michael Romero, Juan Salamanca, Juan Osorio

Temas tratados:

- ¿Qué salió bien? Las pruebas unitarias con Mocktail y los tests de widgets pasaron al primer intento tras correcciones menores.
- ¿Qué salió mal? flutter_animate generaba timers pendientes en los tests de widgets.
- ¿Qué mejorar? En futuros proyectos, diseñar los widgets con testabilidad en mente desde el inicio.

Acuerdos y compromisos:

- Se resuelve el problema de timers con await tester.pump() explícito en cada test.
- Se sube el proyecto a GitHub con .gitignore correcto que excluye venv y .env.
- Se concluye que la arquitectura Flutter + Flask + Firebase + Gemini es robusta y escalable para proyectos educativos móviles.

PRODUCT BACKLOG

Interfaz y Navegación del Juego

ID	Tipo	Actividad	Descripción	Prioridad	SP	Sprint
MC-01	Historia	Splash Screen	Pantalla de carga con logo animado	Alta	2	S1
MC-02	Historia	Login y Registro	Autenticación con Firebase Auth	Alta	5	S1
MC-03	Historia	Home Screen - Edificio	Edificio animado con ventanas de progreso	Alta	8	S2
MC-04	Historia	Mapa de Niveles	Ruta con 8 nodos desbloqueables	Alta	5	S2
MC-05	Historia	Game Screen	Vista primera persona con imagen IBM	Alta	5	S4
MC-06	Historia	Configuración	Switches de audio y créditos	Media	3	S5
MC-07	Historia	Créditos	Pantalla con equipo y universidad	Baja	2	S5

Sistema de Preguntas con IA

ID	Tipo	Actividad	Descripción	Prioridad	SP	Sprint
MC-08	Historia	Endpoint Gemini	Flask genera preguntas con IA por nivel	Alta	8	S3
MC-09	Historia	Validar respuesta	Flask valida y genera retroalimentación	Alta	5	S3
MC-10	Historia	Botón Continuar	Muestra retroalimentación antes de siguiente pregunta	Alta	3	S4
MC-11	Historia	Preguntas en español	Prompt fuerza respuestas en español	Media	2	S4
MC-12	Historia	Fallback de modelos	Sistema usa múltiples modelos Gemini	Alta	3	S4

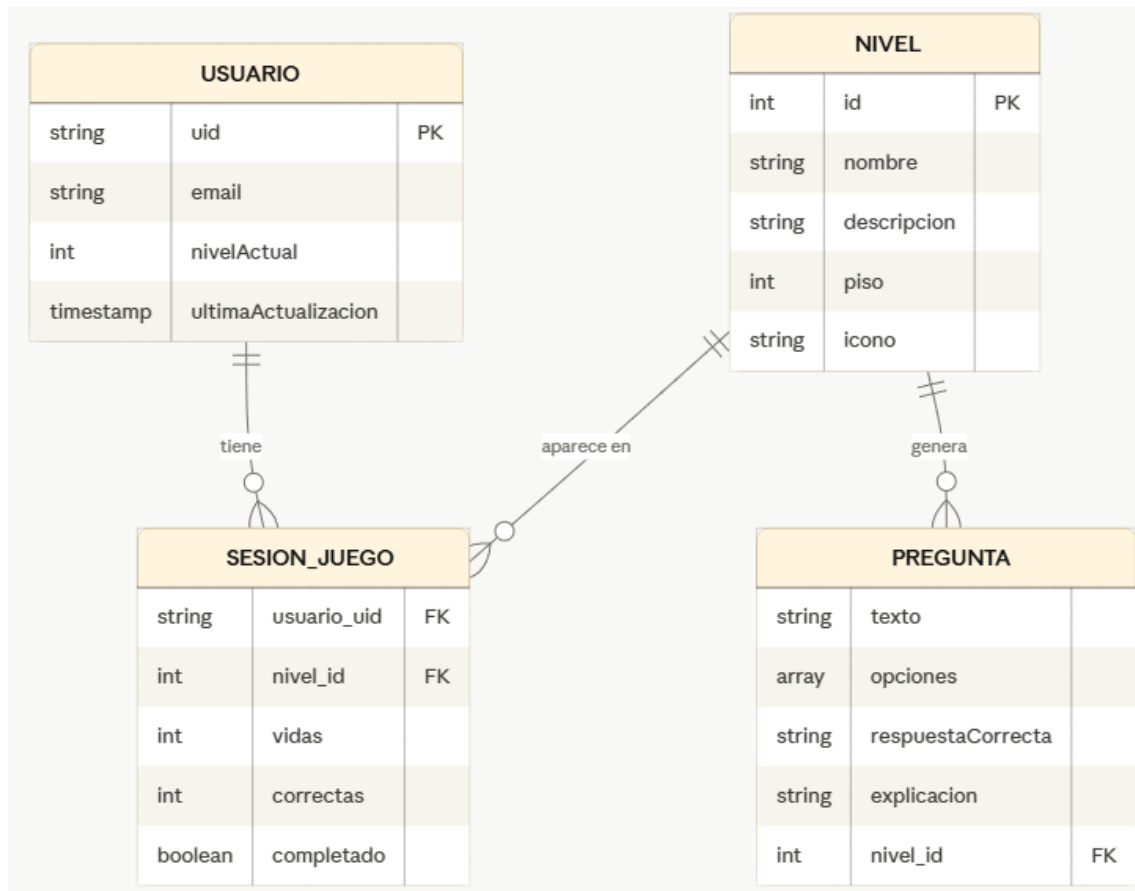
Sistema de Progreso

ID	Tipo	Actividad	Descripción	Prioridad	SP	Sprint
MC-13	Historia	Sistema de vidas	3 corazones por nivel	Alta	3	S4
MC-14	Historia	Game Over	Animación de muerte con retroalimentación	Alta	5	S4
MC-15	Historia	Guardar en Firestore	Progreso guardado en la nube por usuario	Alta	5	S5
MC-16	Historia	Desbloqueo de niveles	Nivel siguiente disponible al completar	Alta	3	S5
MC-17	Historia	Reiniciar progreso	Confirmación y reinicio desde cero	Media	2	S5

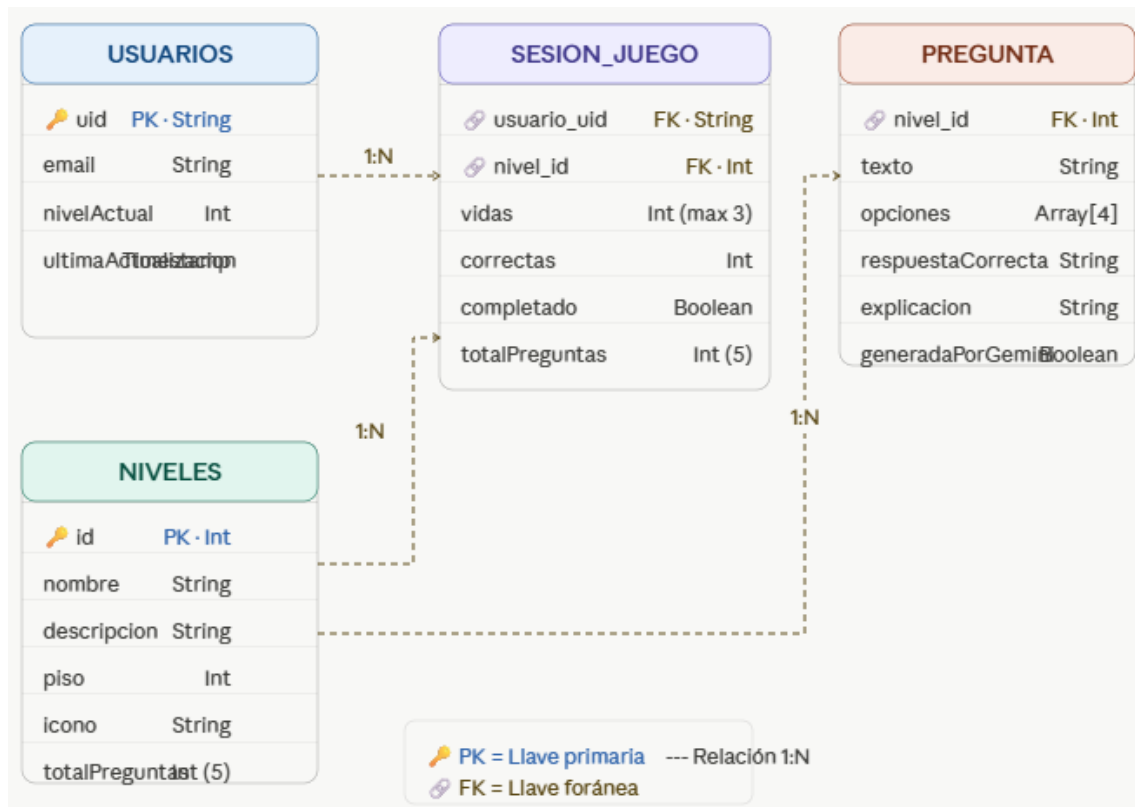
Desarrollo y Calidad

ID	Tipo	Actividad	Descripción	Prioridad	SP	Sprint
MC-18	Tarea	Pruebas HTTP Mocktail	6 pruebas unitarias de GameService	Alta	5	S6
MC-19	Tarea	Pruebas de widgets	13 pruebas de ConfigScreen y CreditsScreen	Alta	5	S6
MC-20	Tarea	Thunder Client	Pruebas manuales de los 3 endpoints Flask	Alta	3	S3
MC-21	Tarea	GitHub	Repositorio privado con README y requirements.txt	Alta	2	S6
MC-22	Tarea	Assets IA	Logo y computador IBM integrados como assets	Media	3	S5

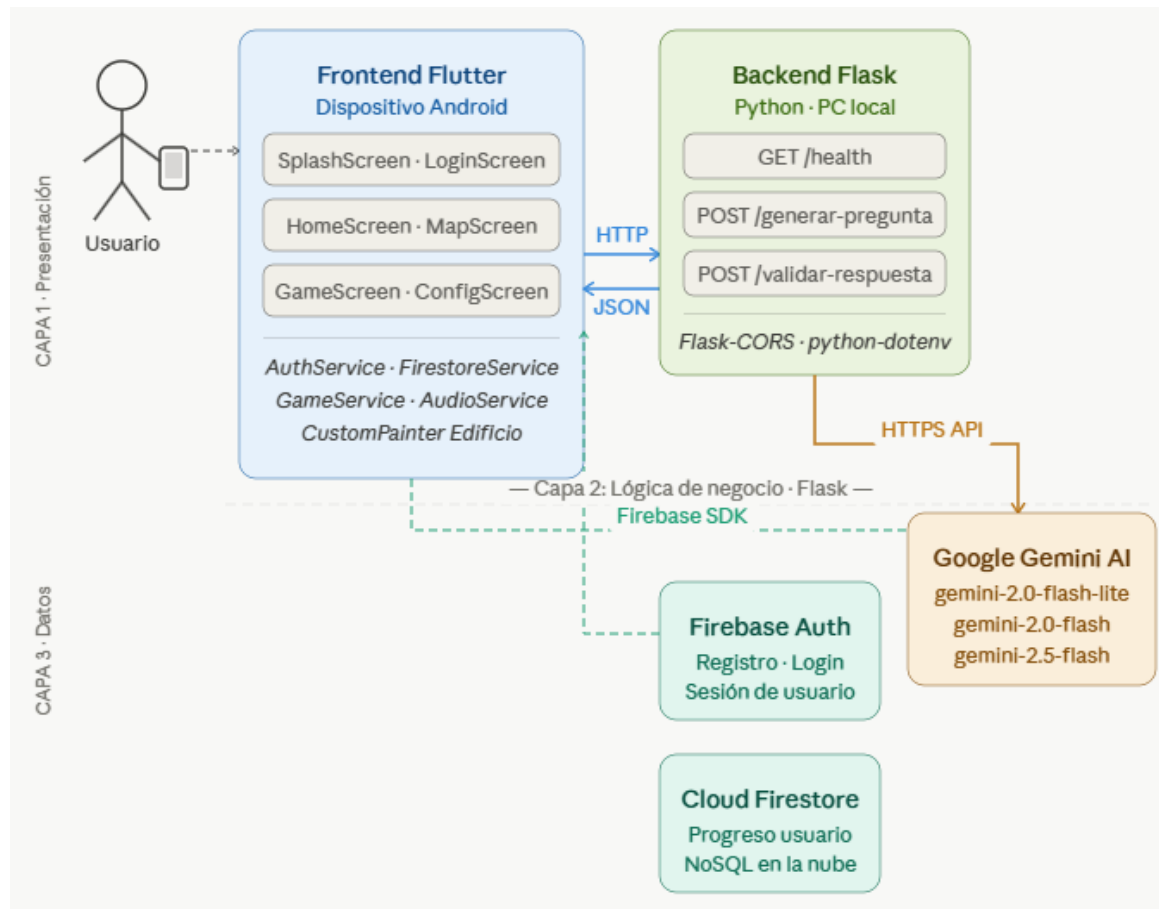
MODELO ENTIDAD-RELACION



MODELO RELACIONAL



DISEÑO DE ARQUITECTURA DE SOFTWARE



PRUEBAS UNITARIAS

Pruebas de Servicios HTTP con Mocktail

Se implementaron 6 pruebas unitarias para el servicio GameService usando el paquete mocktail, que permite simular (mockear) el cliente HTTP sin necesidad de un servidor real.

#	Prueba	Servicio	Resultado esperado	Estado
1	Retorna pregunta correctamente cuando el servidor responde 200	GameService.generarPregunta()	Map con pregunta, opciones y respuesta_correcta	PASS
2	Lanza excepción cuando el servidor responde 500	GameService.generarPregunta()	throwsException	PASS
3	Lanza excepción cuando hay error de conexión	GameService.generarPregunta()	throwsException	PASS
4	Retorna correcto: true cuando la respuesta es correcta	GameService.validarRespuesta()	correcto: true, retroalimentación contiene 'Correcto'	PASS
5	Retorna correcto: false cuando la respuesta es incorrecta	GameService.validarRespuesta()	correcto: false, retroalimentación contiene 'Incorrecto'	PASS
6	Lanza excepción cuando el servidor responde 500	GameService.validarRespuesta()	throwsException	PASS

Resultado: 6/6 pruebas pasando.

Pruebas de Widgets del Frontend

Se implementaron 13 pruebas de widgets para verificar que las pantallas renderizan correctamente sus elementos visuales.

#	Prueba	Pantalla	Elemento verificado	Estado
1	Muestra LinearProgressIndicator de carga	SplashScreen	LinearProgressIndicator	PASS
2	Muestra Scaffold con fondo oscuro	SplashScreen	Scaffold, Text 'MORTAL CODE'	PASS
3	Renderiza con opciones de audio	ConfigScreen	Text 'AUDIO', 'Efectos de sonido', 'Música de fondo'	PASS
4	Contiene dos switches de audio	ConfigScreen	Switch x2	PASS
5	Contiene botón de créditos	ConfigScreen	Text 'CRÉDITOS'	PASS
6	Switches inician en true	ConfigScreen	Switch.value == true	PASS
7	Switch responde al tap y cambia valor	ConfigScreen	Switch.value == false tras tap	PASS
8	Renderiza correctamente	CreditsScreen	CreditsScreen widget exists	PASS
9	Muestra sección de universidad	CreditsScreen	Text contiene 'UNIVERSIDAD'	PASS
10	Muestra sección de equipo	CreditsScreen	Text contiene 'EQUIPO'	PASS
11	Muestra sección de tecnologías	CreditsScreen	Text contiene 'TECNOLOGÍAS'	PASS
12	Muestra sección de agradecimientos	CreditsScreen	Text contiene 'AGRADECIMIENTOS'	PASS
13	Muestra Flutter en tecnologías	CreditsScreen	Text contiene 'Flutter'	PASS

Resultado: 13/13 pruebas pasando. Total del proyecto: 19/19 pruebas unitarias.

Pruebas Manuales con Thunder Client

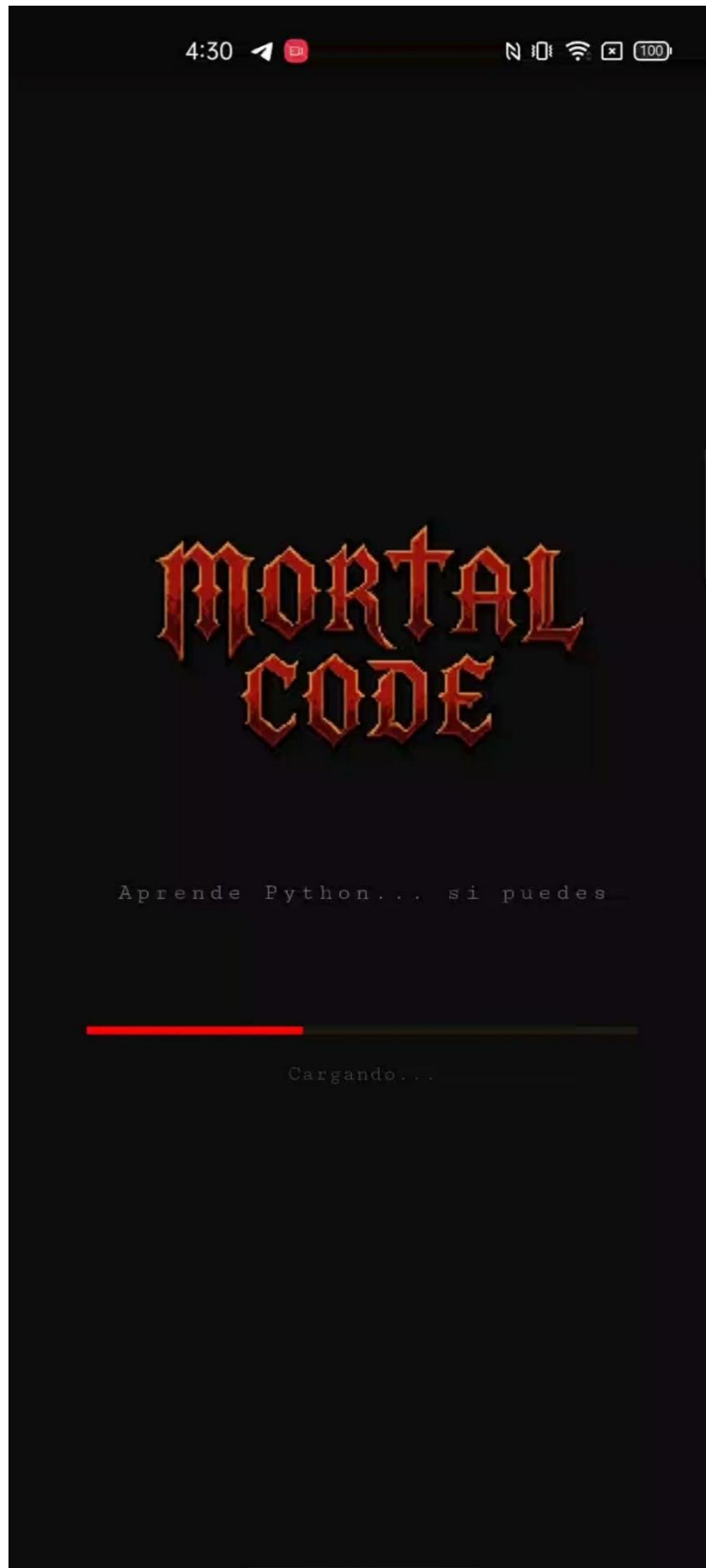
Se realizaron pruebas manuales de los endpoints del backend Flask usando Thunder Client en VS Code:

Endpoint	Método	Body enviado	Respuesta esperada	Estado
/health	GET	(ninguno)	{ status: ok, mensaje: Mortal Code API corriendo }	200 OK
/generar-pregunta	POST	{ nivel: 1 }	JSON con pregunta, opciones, respuesta_correcta, explicacion	200 OK
/validar-respuesta	POST	{ opcion_seleccionada: A, respuesta_correcta: A, ... }	{ correcto: true, retroalimentacion: '...' }	200 OK

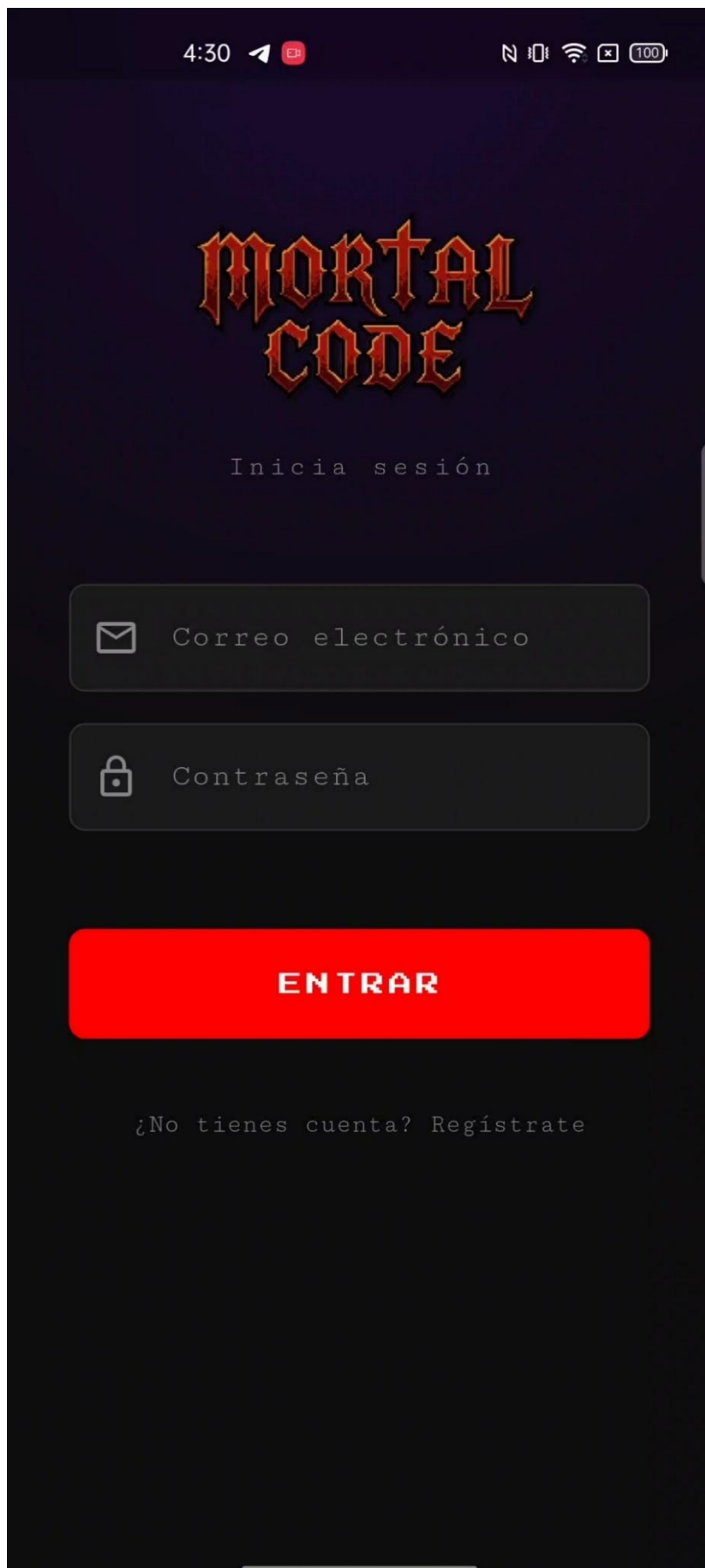
EVIDENCIA

A continuación, se presentan las capturas de pantalla de las principales etapas del desarrollo y pruebas del proyecto:

- Splash Screen con logo animado corriendo en el Realme 6 Pro



- Pantalla de Login y Registro con Firebase Authentication



4:30



MORTAL CODE

Regístrate



Correo electrónico

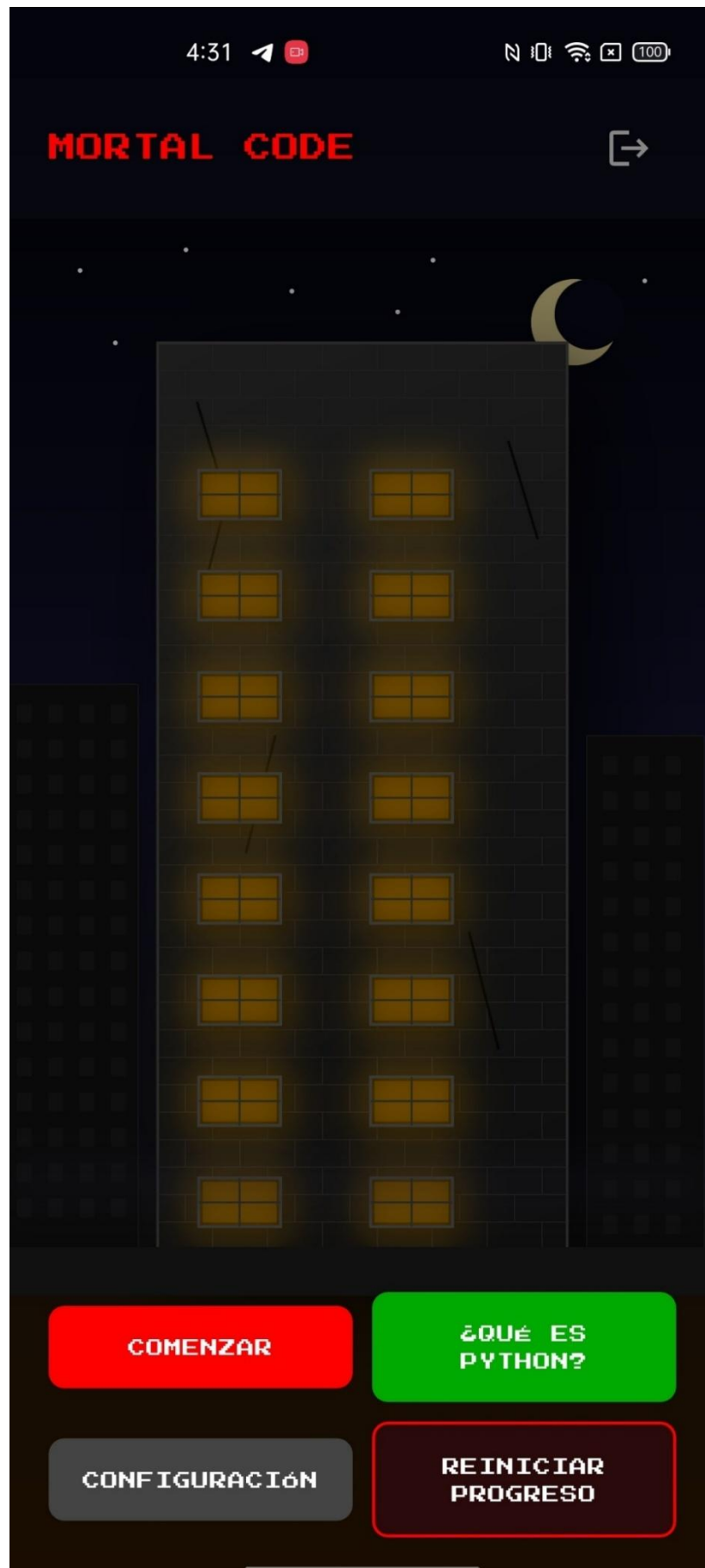


Contraseña

REGISTRARSE

¿Ya tienes cuenta? [Inicia sesión](#)

- Home Screen con el edificio CustomPainter y ventanas iluminadas



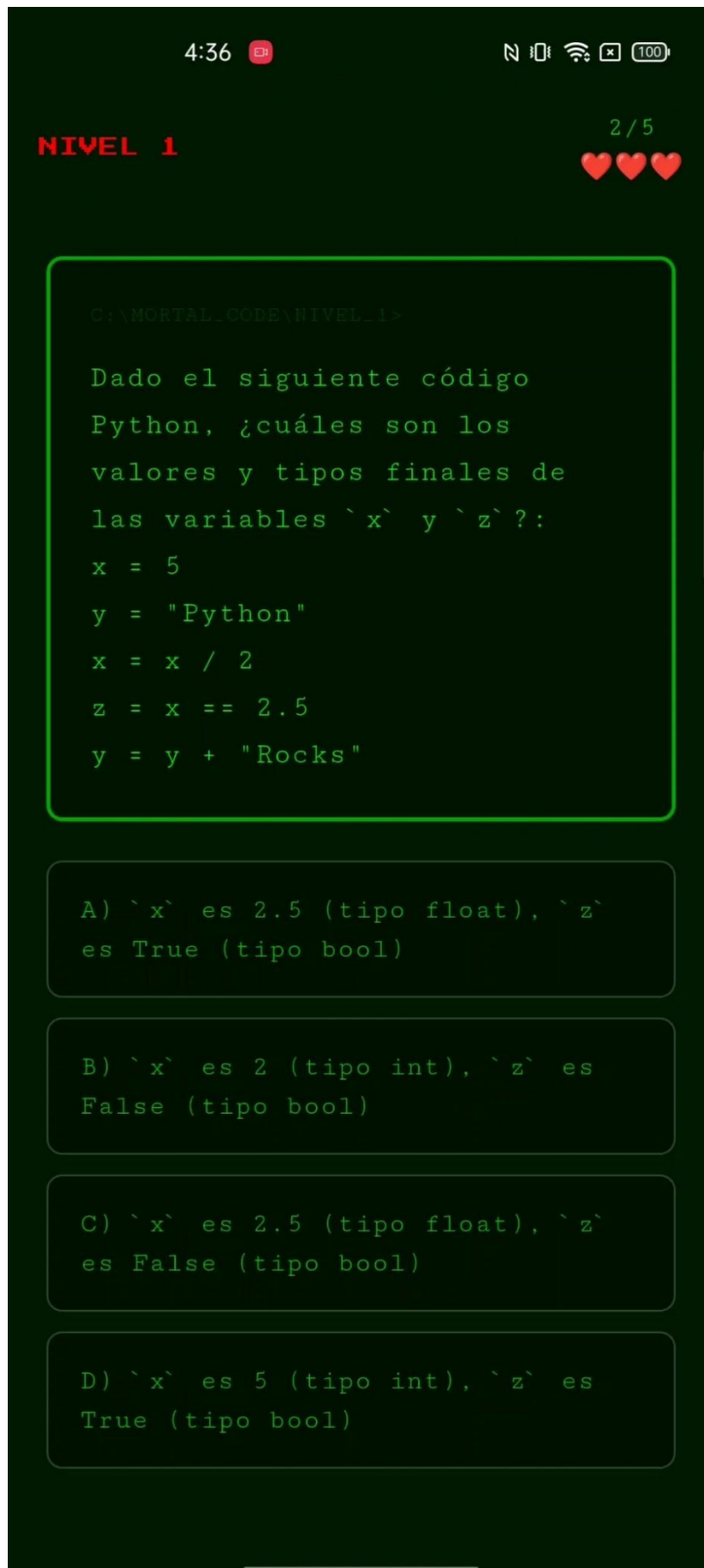
- Mapa de niveles con los 8 nodos



- Game Screen con la imagen del computador IBM en primera persona



- Pantalla de pregunta generada por Gemini con opciones de respuesta



- Retroalimentación positiva con botón CONTINUAR

4:37 

   100

NIVEL 1

2/5

```
y = y + "Rocks"
```

A) ``x`` es 2.5 (tipo float), ``z`` es True (tipo bool)

B) ``x`` es 2 (tipo int), ``z`` es False (tipo bool)

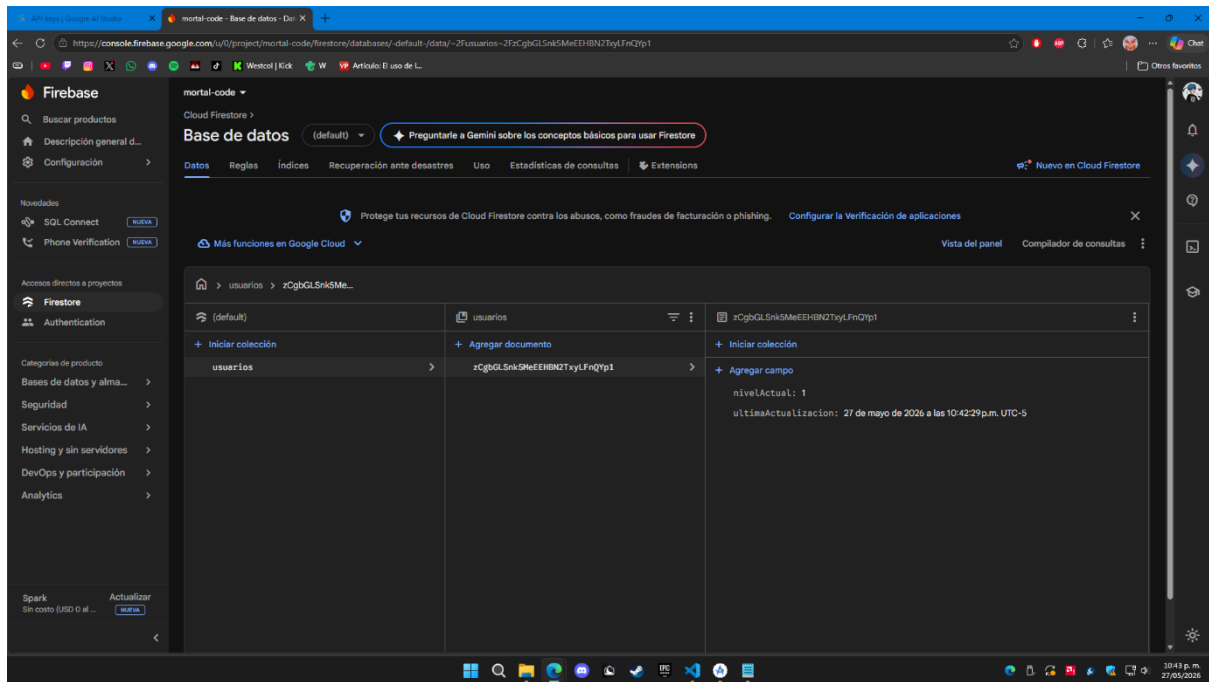
C) ``x`` es 2.5 (tipo float), ``z`` es False (tipo bool)

D) ``x`` es 5 (tipo int), ``z`` es True (tipo bool)

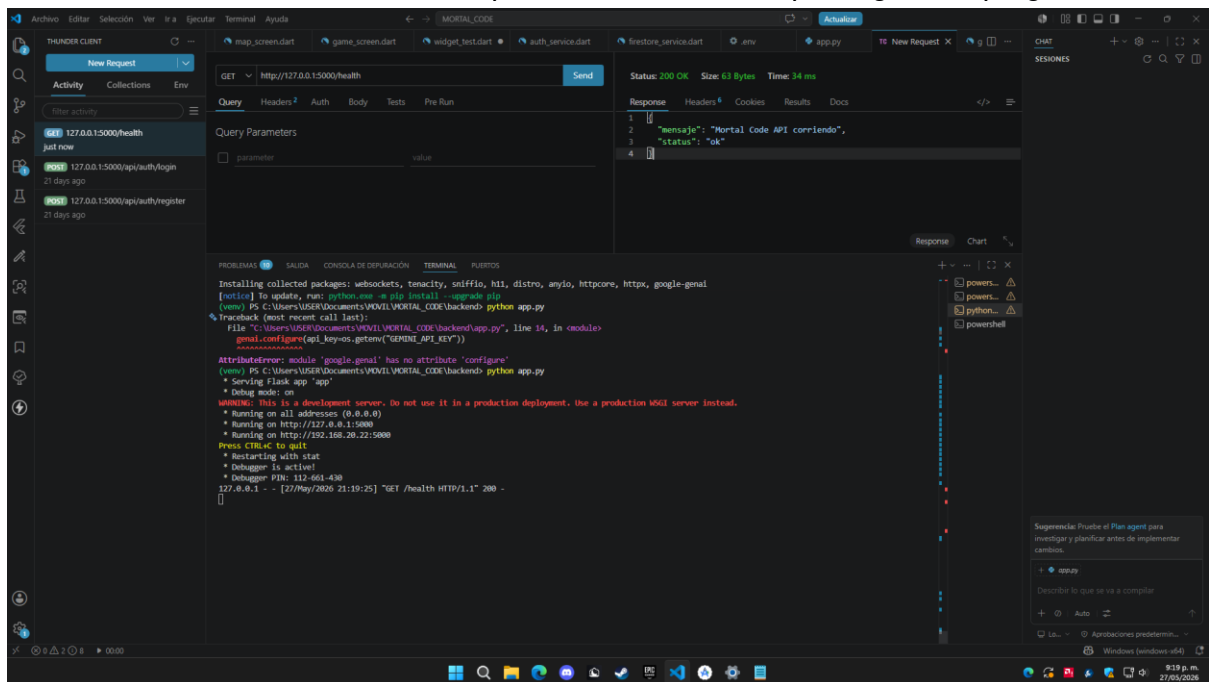
☒ ¡Correcto! Inicialmente, ``x`` es 5 (entero). Al ejecutar ``x` = x / 2``, la división en Python (incluso entre enteros) siempre produce un ``float``, resultando en ``x`` igual a 2.5. Luego, ``z` = x == 2.5`` compara 2.5 con 2.5, lo cual es verdadero, asignando ``True`` a ``z`` (tipo booleano).

CONTINUAR ➔

- Firebase Console mostrando el usuario registrado



- Thunder Client mostrando respuesta 200 OK del endpoint /generar-pregunta



THUNDER CLIENT

New Request

Activity Collections Env

POST 127.0.0.1:5000/health

just now

POST 127.0.0.1:5000/api/auth/login

21 days ago

POST 127.0.0.1:5000/api/auth/register

21 days ago

Query Headers Auth Body Tests Pre Run

Send

Status: 200 OK Size: 1.46 KB Time: 5.77 s

Response Headers Cookies Results Docs

1 {

2 "explicacion": "Python es un lenguaje de tipado dinámico, lo que significa que no se requiere declarar el tipo de una variable de antemano. El intérprete deduce y asigna el tipo de dato automáticamente en función del valor que se le asigna en el momento de la asignación. Las opciones B y C son incorrectas porque Python no requiere declaración explícita de tipos ni restringe el reasignar una variable a un valor de un tipo diferente. La opción D es incorrecta porque 'False' sin comillas es un valor booleano, no una cadena.",

3 "opciones": [

4 "A) Python asigna automáticamente el tipo de dato (entero, cadena, flotante, booleano) a cada variable basándose en el valor que se le proporciona.",

5 "B) Para ejecutar este código, primero se deben declarar explícitamente los tipos de 'num', 'texto', 'pi' y 'es_valido' antes de la asignación.",

6 "C) Una vez que a 'num' se le asigna un valor entero (18), nunca podrá ser reasignado a un valor de otro tipo, como una cadena o un flotante.",

7 "D) La variable 'es_valido' es una cadena de texto porque 'False' es un valor que se puede representar como texto.",

8],

9 "pregunta": "Considera el siguiente código Python:\npython\nnum = 18\ntexto = 'Python'\npi = 3.14\nes_valido = False\n\n¿Cuál de las siguientes afirmaciones es 'correcta' sobre las variables y su asignación en este fragmento de código?",

10 "respuesta_correcta": "A"

11 }

JSON Content

Format

1 {

2 "nivel": 1

3 }

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

C:\Users\USER\Documents\MORTAL_CODE\backend\app.py:3: FutureWarning:

• Debugger is active!

• Debugger PID: 112-661-438

127.0.0.1 - - [27/May/2026 21:23:16] "POST /generar-pregunta HTTP/1.1" 200 -

9:23 p.m.
27/05/2026

THUNDER CLIENT

New Request

Activity Collections Env

POST 127.0.0.1:5000/health

just now

POST 127.0.0.1:5000/api/auth/login

21 days ago

POST 127.0.0.1:5000/api/auth/register

21 days ago

Query Headers Auth Body Tests Pre Run

Send

Status: 200 OK Size: 134 Bytes Time: 9 ms

Response Headers Cookies Results Docs

1 {

2 "correcto": true,

3 "retroalimentacion": "(Correcto En Python las variables se declaran asignando un valor directamente.)"

4 }

JSON Content

Format

1 {

2 "pregunta": "¿Cuál es la forma correcta de declarar una variable en Python?",

3 "opcion_seleccionada": "A",

4 "respuesta_correcta": "A",

5 "explicacion": "En Python las variables se declaran asignando un valor directamente."

6 }

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

C:\Users\USER\Documents\MORTAL_CODE\backend\app.py:3: FutureWarning:

• Debugger is active!

• Debugger PID: 112-661-438

127.0.0.1 - - [27/May/2026 21:23:16] "POST /generar-pregunta HTTP/1.1" 200 -

127.0.0.1 - - [27/May/2026 21:26:23] "POST /validar-respuesta HTTP/1.1" 200 -

9:26 p.m.
27/05/2026

- Terminal mostrando pruebas unitarias pasando (All tests passed!)

```
00/health  TE New Request  TE New Request  map_screen.dart  game_screen.dart  widget_test.dart  widget_screens_test.dart  gignore  widget_screens_test.dart U x
```

```
void main() {  
  group("CreditsScreen", () {  
    // ...  
  });  
}
```

```
Run [Debug]  
testWidgets("muestra sección de agradecimientos", (WidgetTester tester) async {  
  await tester.pumpWidget(  
    const MaterialApp(home: CreditsScreen()),  
  );  
  await tester.pump(const Duration(seconds: 1));  
  await tester.pump(const Duration(seconds: 1));  
  expect(find.textContaining("AGRADECIMIENTOS"), findsOneWidget);  
  timeout: const Timeout(Duration(seconds: 10));  
});
```

```
Run [Debug]  
testWidgets("muestra Flutter en tecnologías", (WidgetTester tester) async {  
  await tester.pumpWidget(  
    const MaterialApp(home: CreditsScreen()),  
  );  
  await tester.pump(const Duration(seconds: 1));  
  await tester.pump(const Duration(seconds: 1));  
  expect(find.textContaining("Flutter"), findsOneWidget);  
  timeout: const Timeout(Duration(seconds: 10));  
});
```

```
PS C:\Users\USER\Documents\MOVIL\VORTAL_CODE\mortal_code> flutter test widget_screens_test.dart  
00:02 +0: All tests passed!  
%PS C:\Users\USER\Documents\MOVIL\VORTAL_CODE\mortal_code>
```

The test description was: switches responden al tap

In run this test again: C:\Users\USER\Documents\MOVIL\Flutter\bin\cache\dart-sdk\bin\dart.exe test C:\Users\USER\Documents\MOVIL\VORTAL_CODE\mortal_code\test\widget_screens_test.dart -p web --plain-name "ConfigScreen switches responden al tap"

Sugerencia: Prueba el Plan Agent para investigar y planificar antes de implementar cambios.

+ widget_screens_test.dart

Describe lo que se va a compilar

+ Auto

Aprobaciones predefinidas...

```
map_screen.dart  game_screen.dart  widget_test.dart  auth_service.dart  firestore_service.dart  env  user_progress.dart  app.py
```

```
1 import 'package:flutter_test/flutter_test.dart';  
2 import 'package:mocktail/mocktail.dart';  
3 import 'package:http/http.dart' as http;  
4 import 'package:mortal_code/services/game_service.dart';  
5  
6 class MockHttpClient extends Mock implements http.Client {}  
7 class MockResponse extends Mock implements http.Response {}  
8 class FakeUri extends Fake implements Uri {}  
9  
10 void main() {  
11   setUpAll(() {  
12     registerFallbackValue(FakeUri());  
13   });  
14  
15   late MockHttpClient mockClient;  
16   late GameService gameService;  
17  
18   setUp(() {  
19     mockClient = MockHttpClient();  
20     gameService = GameService(client: mockClient);  
21   });  
22  
23   group("GameService - generarPregunta", () {  
24     // ...  
25     final mockResponse = MockResponse();  
26     when(() => mockResponse.statusCode).thenReturn(200);  
27     when(() => mockResponse.body).thenReturn("""  
28       {  
29         "pregunta": "¿qué es una variable en python?",  
30         "opciones": ["A) Un dato", "B) Una función", "C) Un módulo", "D) Una clase"],  
31         "respuesta_correcta": "A",  
32         "explicacion": "Una variable almacena datos."  
33       };  
34     """);  
35  
36     when(() => mockClient.post(  
37       any(),  
38       headers: any(named: 'headers'),  
39       body: any(named: 'body'),  
40     )), thenAnswer((_) async => mockResponse);  
41  
42     // ...  
43   });  
44 }  
45
```

```
PS C:\Users\USER\Documents\MOVIL\VORTAL_CODE\mortal_code> flutter test  
PS C:\Users\USER\Documents\MOVIL\VORTAL_CODE\mortal_code> flutter test  
00:01 +0: All tests passed!  
%PS C:\Users\USER\Documents\MOVIL\VORTAL_CODE\mortal_code>
```

Sugerencia: Prueba el Plan Agent para investigar y planificar antes de implementar cambios.

+ widget_test.dart

Describe lo que se va a compilar

+ Auto

Aprobaciones predefinidas...

- Repositorio GitHub con estructura del proyecto

